



Lớp Polynomial — `__call__` + `__add__` + `__str__`

Cài đặt class `Polynomial` đa thức 1 biến. Constructor nhận list/vector hệ số **giảm dần theo bậc**: $[3, 2, 1] = 3x^2 + 2x + 1$. Cần 3 chức năng:

- **In đa thức** dạng đọc được (vd `'3x^2 + 2x + 1'`)
- **Evaluate tại x** — gọi như hàm: `p(x)` trả về số
- **Cộng 2 đa thức** — `p1 + p2` trả về `Polynomial` mới

Quy tắc format khi in:

- Hệ số 0 → bỏ qua (trừ khi tất cả = 0 → in `"0"`)
- Bậc 1 → in `"ax"` (không có `"^1"`)
- Bậc 0 → in số thuần
- Hệ số 1/-1 ở bậc ≥ 1 → bỏ `"1"` (in `x^2` chứ không phải `1x^2`; in `-x` chứ không phải `-1x`)
- Các term nối bằng `" + "` hoặc `" - "` (có space 2 bên)

Python: override `__str__`, `__call__(self, x)` (để gọi `p(x)`), `__add__(self, other)`.

C++: overload friend `std::ostream& operator<<, int operator()(int x) const (functor), Polynomial operator+(const Polynomial&) const`. Khi cộng 2 đa thức khác bậc, zero-pad đa thức ngắn hơn rồi cộng từng phần tử.

Bạn chỉ nộp khai báo lớp

File `.py` bạn nộp chỉ chứa **Polynomial** — **không** viết `main`, `if __name__`, hay driver đọc stdin. Grader **lấy tên file bạn nộp** làm tên module, **import lớp** của bạn, **chạy driver** để ném stdin vào và so sánh stdout.

 **Đặt tên file** là tên identifier hợp lệ (vd. `Point.py`, `TuLanh.py`, `Polynomial.py`). Nếu đặt bậy (`my-bai.py`, `1.py`) grader fallback `student_solution`.

Luồng chấm bài

`Polynomial.py` (bạn nộp) → `from Polynomial import *` → driver grader → stdin → stdout → so khớp = pass

 Khung lớp bạn nộp lên

PythonC++

```
class Polynomial:
    ...
    def __init__(self, coeffs: list):
```

```

# coeffs[0] là hệ số' bậc cao nhất

self.coeffs = list(coeffs)

# TODO: __str__ → vd '3x^2 + 2x + 1' (nhớ handle hệ số' 0 và dấu -)

# - hệ số' 0 thì bỏ qua

# - bậc 1 in là 'ax' (không có '^1')

# - bậc 0 in là số' thuần

# - toàn bộ hệ số' = 0 thì in '0'

#

# TODO: __call__(self, x) – evaluate bằng Horner's method

# TODO: __add__(self, other) – align độ dài, cộng từng hệ số'

```

```

#include <vector>

```

```

#include <sstream>

```

```

class Polynomial {

```

```

    std::vector<int> coeffs_; // coeffs_[0] = bậc cao nhất

```

```

public:

```

```

    Polynomial(const std::vector<int>& c) : coeffs_(c) {}

```

```

    // TODO: int operator()(int x) const – evaluate Horner's method.

```

```

    int operator()(int x) const { return 0; }

```

```

// TODO: Polynomial operator+(const Polynomial& o) const –

//      align by zero-pad coeffs ngắn hơn rồi cộng từng phân tử?

Polynomial operator+(const Polynomial& o) const { return *this; }

// TODO: friend std::ostream& operator<<(std::ostream&, const Polynomial&)

//      Format giống Python: '3x^2 + 2x + 1'. Quy tắc:

//      - hệ số 0 thì bỏ qua (trừ khi tất cả 0 → in '0')

//      - bậc 1 in là 'ax' (không có '^1')

//      - bậc 0 in là số 'thuần'

//      - dấu '+' / '-' join các term, hệ số 1 và -1 ở bậc >= 1 bỏ '1'

friend std::ostream& operator<<(std::ostream& os, const Polynomial& p);

};

std::ostream& operator<<(std::ostream& os, const Polynomial& p) {

    // TODO: implement

    return os;

}

```

▣ Grader sẽ làm gì với lớp của bạn

1. Đọc 2 dòng hệ số (số nguyên, cách nhau space), tạo `p1` và `p2`
2. Đọc `x` (int)
3. In `p1`, `p2`, `p1 + p2` — Py: `print(p1)` / C++: `std::cout << p1`

4. In giá trị $p_1(x)$ rồi $p_2(x)$ — Py: `print(p1(x))` (gọi `__call__`) / C++: `std::cout << p1(x)` (gọi `operator()`)

► Xem code driver grader tự chạy

Grader chạy đoạn code sau cùng file bạn nộp. Dùng nút ở khung khai báo để xem driver cho từng ngôn ngữ:

```
from Polynomial import * # file SV nộp

c1 = list(map(int, input().split()))

c2 = list(map(int, input().split()))

x = int(input())

p1 = Polynomial(c1)

p2 = Polynomial(c2)

print(p1)

print(p2)

print(p1 + p2)

print(p1(x))

print(p2(x))

#include <iostream> // grader tự prepend khi compile

// ... + class bạn viết ...

int main() {

    auto read_coeffs = []() {
```

```

        std::string line; std::getline(std::cin, line);

        std::vector<int> v; std::stringstream ss(line); int x;

        while (ss >> x) v.push_back(x);

        return v;

};

auto c1 = read_coeffs();

auto c2 = read_coeffs();

int x; std::cin >> x;

Polynomial p1(c1), p2(c2);

std::cout << p1 << std::endl;

std::cout << p2 << std::endl;

std::cout << (p1 + p2) << std::endl;

std::cout << p1(x) << std::endl;

std::cout << p2(x) << std::endl;

return 0;

}

```

□ Testcase mẫu — để đối chiếu format

Grader chạy *code SV + driver* rồi so sánh stdout từng dòng (strip whitespace cuối, bỏ dòng trống thừa ở cuối). Các testcase ẩn có cùng định dạng.

Ví dụ 1

stdin

```
1 5
```

```
2
```

expected stdout

```
3x^2 + 2x + 1
```

```
x + 5
```

```
3x^2 + 3x + 6
```

```
17
```

```
7
```

Ví dụ 2

stdin

```
1 0 -1
```

```
1 0 1
```

```
3
```

expected stdout

```
x^2 - 1
```

```
x^2 + 1
```

```
2x^2
```

```
8
```

```
10
```

Ngôn ngữ hỗ trợ: bài này nhận `.py/ .cpp`. Grader tự ghép driver + class bạn nộp rồi chạy. Java sẽ mở khi có driver tương ứng.

□

Đây là bài **thực hành** — bạn **nộp lại bao nhiêu lần cũng được**. Mỗi lần nộp, hệ thống giữ điểm cao nhất. Thử, đọc error, fix, thử tiếp.